

Laboratorio · Gestione delle sequenze e analisi della qualità

Statistical Models for Genetics · 2 ore

Modulo integrativo · companion del .pptx Lezione NGS

2026-05-19

Contents

Premessa · cosa faremo	1
Blocco A · FASTQ in R, distribuzione empirica della qualità	2
A.1 · Il formato, ricapitolato in tre righe	2
A.2 · Generiamo un FASTQ da zero (per controllo)	2
A.3 · Distribuzione di Q per posizione: il classico boxplot di FastQC	4
A.4 · Esercizio autonomo · 10 min	5
Blocco B · Phred score, modelli statistici dietro e calibrazione	6
B.1 · La formula e cosa significa davvero	6
B.2 · Costruiamo un piccolo “ground truth” e iniettiamo errori	7
B.3 · Test di calibrazione	8
B.4 · Esercizio · un basecaller miscalibrato	9
Blocco C · Copertura, bias GC, e la favola della Poisson (30 min)	11
C.1 · La favola	11
C.2 · Simuliamo un mini-genoma con regioni GC eterogenee	12
C.3 · Frammentiamo con bias di efficienza PCR	13
C.4 · Il test: la copertura è Poissoniana?	15
C.5 · Esercizio · 10 min	15
Blocco D · Lettura critica di un report MultiQC reale (30 min)	16
D.1 · Il report del corso	16
D.2 · Le 7 sezioni che dovete saper leggere	16
D.3 · Esercizio guidato	17
D.4 · La domanda finale, statistica	17
Sintesi · cosa abbiamo imparato	18
Per chi vuole approfondire	18

Premessa · cosa faremo

In due ore lavoriamo sui dati come li ha visti il sequenziatore, non come arrivano nel VCF. L'obiettivo non è imparare i comandi: è capire **dove vive il rumore nel FASTQ** e **come si modella statisticamente**.

Il laboratorio è organizzato in quattro blocchi di 30 minuti:

Blocco	Tema	Dati	Output statistico
A	FASTQ in R: struttura e parsing	Simulati (controllo del ground truth)	Distribuzione empirica della qualità
B	Phred score: cos'è e quanto è calibrato	Simulati con errori iniettati	Test di calibrazione
C	Bias di copertura e GC	Simulati su mini-genoma	Test vs. Poisson
D	Lettura critica di MultiQC	multiqc_report.html del corso	Finii di QC giustificati

Filo conduttore statistico: il FASTQ non è il dato grezzo, è già una stima rumorosa. Ogni metrica di QC che calcoliamo è un test su un'ipotesi nulla che spesso non viene esplicitata. Oggi le esplicitiamo.

Pacchetti necessari. Se Biostrings non c'è, le sezioni che lo usano hanno fallback in R base.

```
suppressPackageStartupMessages({
  library(ggplot2)
  library(dplyr)
  library(tidyr)
})

has_biostrings <- requireNamespace("Biostrings", quietly = TRUE)
has_shortread <- requireNamespace("ShortRead", quietly = TRUE)

if (!has_biostrings) {
  message("Biostrings non installato. Useremo solo R base.")
  message("Per installarlo: BiocManager::install('Biostrings')")
}
```

Blocco A · FASTQ in R, distribuzione empirica della qualità

A.1 · Il formato, ricapitolato in tre righe

Un record FASTQ ha **sempre quattro righe**, sempre in quest'ordine:

```
@READ_ID          ← header (inizia con @)
GATCAAATAGTAAATCCATTGTTCAAC  ← sequenza
+                  ← separatore
! '*((((***+))%%%++) (%%%) .1***-+* ' ← qualità (ASCII)
```

La riga 4 ha **esattamente la stessa lunghezza** della riga 2. Ogni carattere ASCII codifica un Phred score:
 $Q = \text{ASCII} - 33$ (codifica Phred+33).

A.2 · Generiamo un FASTQ da zero (per controllo)

Per il primo esercizio non scarichiamo nulla: generiamo un FASTQ simulato dove **sappiamo la verità**. È l'unico modo per testare se le nostre metriche fanno quello che pensiamo.

```
#' Simula un FASTQ Illumina-like con profilo di qualità realistico
#
#' @param n_reads numero di read
#' @param read_len lunghezza di ogni read
```

```

#' @param q_profile vettore di lunghezza read_len con il Phred score MEDIO per posizione
#' @param q_sd deviazione standard del Q per posizione (rumore tra read)
simulate_fastq <- function(n_reads = 1000, read_len = 100,
                           q_profile = NULL, q_sd = 3) {
  if (is.null(q_profile)) {
    # Profilo a campana sbilenca caratteristico di Illumina:
    # parte da Q32, sale al picco ~Q38 attorno alla posizione 20-30, decresce a Q28 a fine read
    pos <- seq_len(read_len)
    q_profile <- 38 - 6 * dnorm(pos, mean = 5, sd = 8) / dnorm(5, 5, 8) -
      (pos / read_len)^2 * 10
    q_profile <- pmax(2, pmin(40, q_profile))
  }

  # Genera Q sample-by-sample come rumore gaussiano attorno al profilo
  Q <- matrix(
    pmax(2, pmin(40, round(rnorm(n_reads * read_len,
                                  mean = rep(q_profile, each = n_reads),
                                  sd = q_sd))))),
    nrow = n_reads, ncol = read_len
  )

  # Genera sequenze A/C/G/T uniformi (per ora ignoriamo il bias di sequenza)
  bases <- matrix(sample(c("A", "C", "G", "T"), n_reads * read_len, replace = TRUE),
                  nrow = n_reads, ncol
                    = read_len)

  list(
    seq = apply(bases, 1, paste0, collapse = ""),
    qual_matrix = Q, # n_reads x read_len
    qual_ascii = apply(Q, 1, function(q) intToUtf8(q + 33L)) # codifica Phred+33
  )
}

sim <- simulate_fastq(n_reads = 5000, read_len = 100)
str(sim, max.level = 1, list.len = 3)

```

```

## List of 3
## $ seq      : chr [1:5000] "ACAGCACGGCAATTTCCGCGTATGGCCGGCCTCGACGCCAGGTAGATGTCCATATCTTCGCGGGTGCCT"
## $ qual_matrix: num [1:5000, 1:100] 37 31 34 35 34 32 37 32 39 33 ...
## $ qual_ascii : chr [1:5000] "FBE>?@C?BBDBDF@CBBBFEHEEEIIEAIGFFDAFFFBIDBDEDEFFDG?FDHC?E@DFGC@CF=?ADB"

```

Diamo un'occhiata al primo record nel formato FASTQ vero:

```

cat("@SIM_READ_1", "\n",
    sim$seq[1], "\n",
    "+", "\n",
    sim$qual_ascii[1], "\n", sep = "")

```

```

## @SIM_READ_1
## ACAGCACGGCAATTTCCGCGTATGGCCGGCCTCGACGCCAGGTAGATGTCCATATCTTCGCGGGTGCCTGGTTCACATGTCGCACCAGGCTGACACCGA
## +
## FBE>?@C?BBDBDF@CBBBFEHEEEIIEAIGFFDAFFFBIDBDEDEFFDG?FDHC?E@DFGC@CF=?ADBCE;@CC>=CAABADCA>F:=B>=>=?;??=

```

Domanda 1. La quarta riga sembra una bestemmia da fumetto o sembra leggibile? Cosa state guardando?

Risposta attesa: i caratteri sono prevalentemente alfabetici maiuscoli (A-I) e simboli alti come ?,

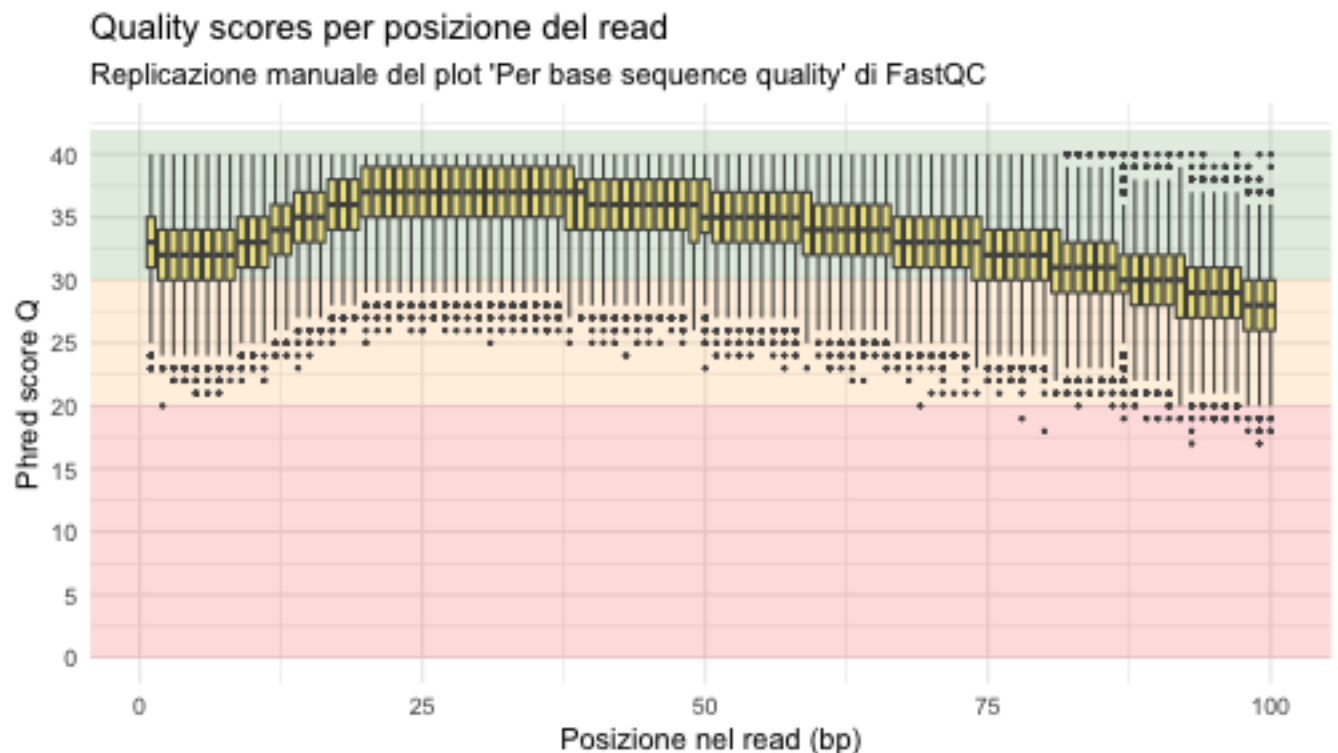
>, =. Sono Q score nel range 28-40, cioè alta qualità. Se vedeste prevalentemente !"#%&, sareste in zona Q < 10.

A.3 · Distribuzione di Q per posizione: il classico boxplot di FastQC

Questo è il primo plot che troverete in ogni report. Lo costruiamo a mano per capire cosa stiamo guardando.

```
qual_long <- as.data.frame(sim$qual_matrix) |>
  setNames(seq_len(ncol(sim$qual_matrix))) |>
  pivot_longer(everything(), names_to = "position", values_to = "Q") |>
  mutate(position = as.integer(position))

ggplot(qual_long, aes(x = position, y = Q, group = position)) +
  # Bande di colore stile FastQC: rosso < 20, giallo 20-30, verde > 30
  annotate("rect", xmin = -Inf, xmax = Inf, ymin = 0, ymax = 20, alpha = 0.15, fill = "red") +
  annotate("rect", xmin = -Inf, xmax = Inf, ymin = 20, ymax = 30, alpha = 0.15, fill = "orange") +
  annotate("rect", xmin = -Inf, xmax = Inf, ymin = 30, ymax = 42, alpha = 0.15, fill = "forestgreen") +
  geom_boxplot(outlier.size = 0.3, fill = "khaki", colour = "grey30") +
  scale_y_continuous(breaks = seq(0, 40, 5), limits = c(0, 42)) +
  labs(title = "Quality scores per posizione del read",
       subtitle = "Replicazione manuale del plot 'Per base sequence quality' di FastQC",
       x = "Posizione nel read (bp)", y = "Phred score Q") +
  theme_minimal(base_size = 11)
```



Discussione (5 min). Cosa **non** è MCAR (Missing Completely At Random) in questo grafico?

La qualità per posizione non è indipendente dalla posizione. Q dipende sistematicamente dalla posizione lungo il read: questo è phasing/dephasing dei cluster Illumina. Per i modelli a valle implica che gli errori non sono i.i.d., e qualunque test che assume i.i.d. va calibrato di conseguenza.

A.4 · Esercizio autonomo · 10 min

Costruite **due** alternative al boxplot di FastQC:

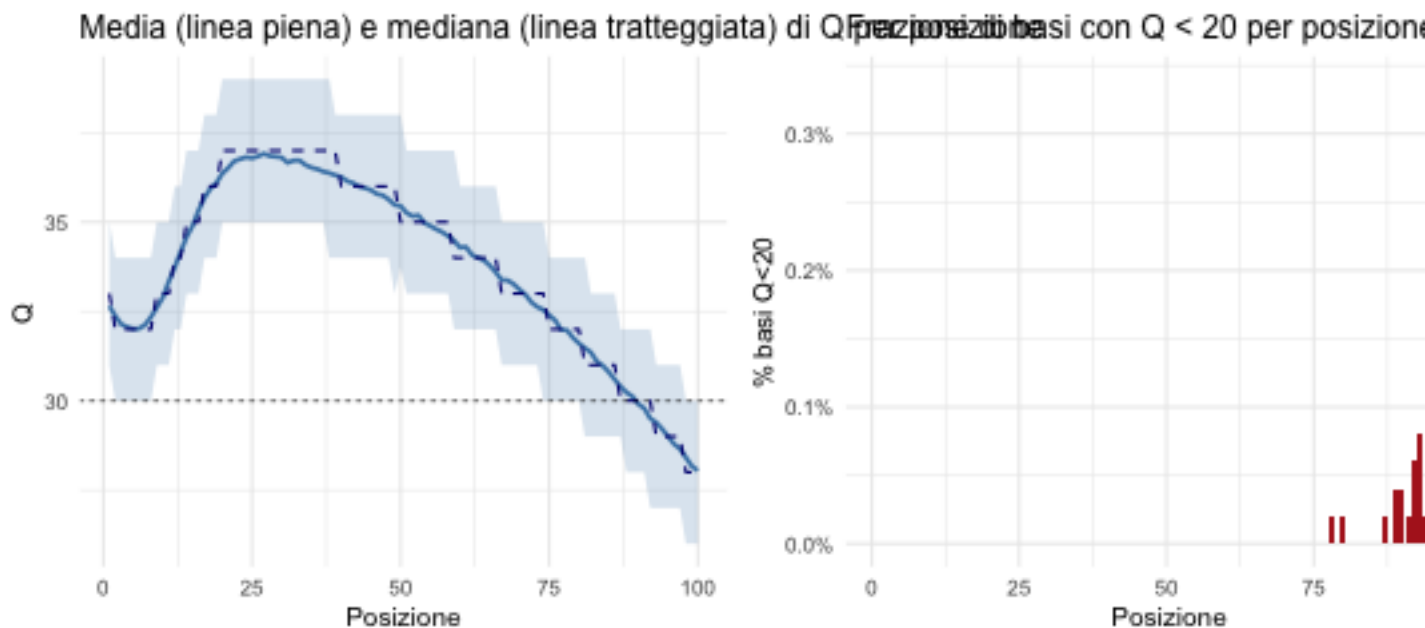
1. Un grafico a linee della **media** e della **mediana** di Q per posizione, con banda interquartile.
2. La **frazione di basi con Q < 20** per posizione (Q < 20 = la zona rossa di FastQC).

```
# Soluzione (mostrarla solo dopo la discussione)
qual_summary <- qual_long |>
  group_by(position) |>
  summarise(
    Q_mean   = mean(Q),
    Q_median = median(Q),
    Q_q25    = quantile(Q, 0.25),
    Q_q75    = quantile(Q, 0.75),
    pct_lowQ = mean(Q < 20),
    .groups  = "drop"
  )

p1 <- ggplot(qual_summary, aes(position)) +
  geom_ribbon(aes(ymin = Q_q25, ymax = Q_q75), alpha = 0.2, fill = "steelblue") +
  geom_line(aes(y = Q_mean), colour = "steelblue", linewidth = 0.8) +
  geom_line(aes(y = Q_median), colour = "navy", linewidth = 0.5, linetype = "dashed") +
  geom_hline(yintercept = 30, linetype = "dotted") +
  labs(title = "Media (linea piena) e mediana (linea tratteggiata) di Q per posizione",
       y = "Q", x = "Posizione") + theme_minimal(base_size = 10)

p2 <- ggplot(qual_summary, aes(position, pct_lowQ)) +
  geom_col(fill = "firebrick") +
  scale_y_continuous(labels = scales::percent) +
  labs(title = "Frazione di basi con Q < 20 per posizione",
       y = "% basi Q<20", x = "Posizione") + theme_minimal(base_size = 10)

# Mostriamo affiancati (richiede patchwork se installato, altrimenti uno alla volta)
if (requireNamespace("patchwork", quietly = TRUE)) {
  patchwork::wrap_plots(p1, p2, ncol = 2)
} else {
  print(p1); print(p2)
}
```



Punto statistico. Quando nel Modulo 2 applicherete un filtro come `qual_threshold = 20` sulle basi, state implicitamente assumendo che `mean(Q < 20)` sia indipendente da posizione, campione e contesto di sequenza. Le prime due sono **falsificabili oggi**, prima ancora di fare GWAS.

Blocco B · Phred score, modelli statistici dietro e calibrazione

B.1 · La formula e cosa significa davvero

Il Phred score è definito come

$$Q = -10 \cdot \log_{10}(P_{\text{errore}})$$

Quindi $P_{\text{errore}} = 10^{-Q/10}$.

```
phred_table <- tibble::tibble(
  Q = c(10, 20, 30, 40),
  P_errore = 10^(-Q/10),
  Accuratezza = 1 - P_errore,
  Errori_su_1Mb = round(P_errore * 1e6)
)
phred_table
```

```
## # A tibble: 4 x 4
##       Q P_errore Accuratezza Errori_su_1Mb
##   <dbl>   <dbl>     <dbl>     <dbl>
## 1    10    0.1      0.9      100000
## 2    20   0.01    0.99     10000
## 3    30  0.001    0.999     1000
## 4    40 0.0001    1.000      100
```

Punto cruciale. Q è una **predizione probabilistica fatta dal basecaller**, non una misura diretta. Il basecaller può sbagliare. Per verificarlo serve confrontarlo con la verità — la **calibrazione** del classificatore.

B.2 · Costruiamo un piccolo “ground truth” e iniettiamo errori

Procediamo così: prendiamo una “verità” nota (una sequenza di riferimento), simuliamo read con errori iniettati a tasso noto, e vediamo se i Q dichiarati matchano gli errori reali.

```
# Genera reads con errori iniettati a tasso specificato dal Q dichiarato
#
# @param truth sequenza di riferimento
# @param n_reads numero di reads da generare
# @param read_len lunghezza dei reads
# @return data.frame con read, qualità dichiarate, e bandiere di errore reale
simulate_with_errors <- function(truth, n_reads = 10000, read_len = 50) {
  truth_chars <- strsplit(truth, "")[[1]]
  if (length(truth_chars) < read_len) stop("truth troppo corta")

  bases <- c("A", "C", "G", "T")

  out <- vector("list", n_reads)
  for (i in seq_len(n_reads)) {
    # campiona posizione di partenza random sulla truth
    start <- sample.int(length(truth_chars) - read_len + 1, 1)
    true_seq <- truth_chars[start:(start + read_len - 1)]

    # ogni base ha un Q assegnato dal basecaller. Distribuzione uniforme tra Q15 e Q40
    # (per esplorare tutto il range di calibrazione)
    Q_declared <- sample(15:40, read_len, replace = TRUE)
    P_err <- 10^(-Q_declared / 10) #cambiare carattere!!

    # iniettiamo gli errori in modo onesto: il basecaller dichiara Q correttamente
    # questa è la condizione di CALIBRAZIONE PERFETTA
    is_error <- runif(read_len) < P_err
    read_seq <- true_seq
    # sostituisci con una base diversa scelta a caso
    if (any(is_error)) {
      for (j in which(is_error)) {
        read_seq[j] <- sample(setdiff(bases, true_seq[j]), 1)
      }
    }

    out[[i]] <- data.frame(
      read_id = i,
      position = seq_len(read_len),
      true_base = true_seq,
      read_base = read_seq,
      Q = Q_declared,
      is_error = is_error
    )
  }
  do.call(rbind, out)
}

# Verità: una sequenza random di 5 kb (potrebbe essere un piccolo genoma batterico)
```

```
truth_seq <- paste(sample(c("A","C","G","T"), 5000, replace = TRUE), collapse = "")

calib_data <- simulate_with_errors(truth_seq, n_reads = 5000, read_len = 50)
head(calib_data, 8)
```

```
##   read_id position true_base read_base  Q is_error
## 1      1         1         G      G 32  FALSE
## 2      1         2         T      T 19  FALSE
## 3      1         3         C      C 26  FALSE
## 4      1         4         C      C 16  FALSE
## 5      1         5         G      G 27  FALSE
## 6      1         6         A      A 32  FALSE
## 7      1         7         A      A 38  FALSE
## 8      1         8         G      G 33  FALSE
```

```
aggregate(is_error ~ Q, data = calib_data, mean)
```

```
##      Q is_error
## 1  15 0.0305140
## 2  16 0.0262366
## 3  17 0.0196324
## 4  18 0.0184390
## 5  19 0.0113945
## 6  20 0.0093324
## 7  21 0.0077878
## 8  22 0.0060073
## 9  23 0.0055301
## 10 24 0.0037543
## 11 25 0.0023829
## 12 26 0.0027228
## 13 27 0.0016906
## 14 28 0.0013741
## 15 29 0.0014612
## 16 30 0.0016543
## 17 31 0.0011401
## 18 32 0.0003123
## 19 33 0.0001029
## 20 34 0.0006281
## 21 35 0.0004137
## 22 36 0.0001023
## 23 37 0.0002083
## 24 38 0.0001030
## 25 39 0.0000000
## 26 40 0.0001031
```

B.3 · Test di calibrazione

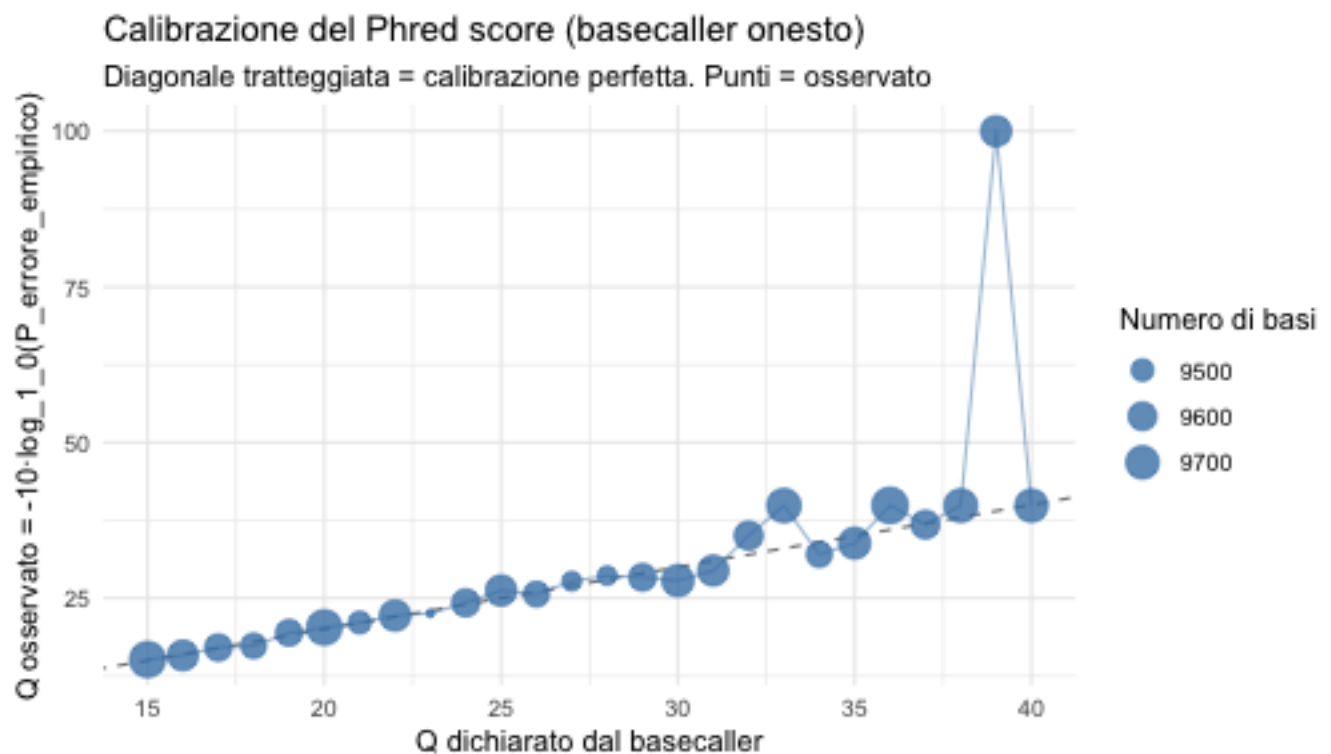
```
# Per ogni Q dichiarato, calcoliamo il tasso di errore osservato
calib_summary <- calib_data |>
  group_by(Q) |>
  summarise(
    n_basi      = n(),
    n_errori    = sum(is_error),
    P_obs       = n_errori / n_basi,
```

```

    .groups      = "drop"
  ) |>
  mutate(
    P_declared = 10^(-Q / 10),
    Q_observed = -10 * log10(pmax(P_obs, 1e-10))
  )

ggplot(calib_summary, aes(Q, Q_observed)) +
  geom_abline(slope = 1, intercept = 0, colour = "grey50", linetype = "dashed") +
  geom_point(aes(size = n_basi), colour = "steelblue", alpha = 0.8) +
  geom_line(colour = "steelblue", alpha = 0.5) +
  labs(
    title = "Calibrazione del Phred score (basecaller onesto)",
    subtitle = "Diagonale tratteggiata = calibrazione perfetta. Punti = osservato",
    x = "Q dichiarato dal basecaller",
    y = "Q osservato = -10·log10(P_errore_empirico)",
    size = "Numero di basi"
  ) +
  theme_minimal(base_size = 11)

```



I punti dovrebbero allinearsi sulla diagonale (con rumore campionario, soprattutto per Q alti dove gli errori osservati sono pochi). Adesso facciamo l'esercizio cattivo.

B.4 · Esercizio · un basecaller miscalibrato

Modificate `simulate_with_errors` per simulare un basecaller **ottimista** che dichiara $Q = Q_{\text{vero}} + 5$ (sopravaluta la qualità di 5 unità). Rifate il plot di calibrazione e quantificate **di quanto il basecaller mente**.

```

simulate_with_errors_biased <- function(truth, n_reads = 5000, read_len = 50, bias = 5) {
  truth_chars <- strsplit(truth, "")[[1]]
  bases <- c("A", "C", "G", "T")
  out <- vector("list", n_reads)
  for (i in seq_len(n_reads)) {
    start <- sample.int(length(truth_chars) - read_len + 1, 1)
    true_seq <- truth_chars[start:(start + read_len - 1)]

    # Q VERO (interno) random uniforme 10-35
    Q_true <- sample(10:35, read_len, replace = TRUE)
    P_err <- 10^(-Q_true / 10)
    is_error <- runif(read_len) < P_err

    # Il basecaller DICHIARA Q_true + bias (è ottimista)
    Q_declared <- pmin(40, Q_true + bias)

    read_seq <- true_seq
    if (any(is_error)) {
      for (j in which(is_error)) {
        read_seq[j] <- sample(setdiff(bases, true_seq[j]), 1)
      }
    }
    out[[i]] <- data.frame(read_id = i, position = seq_len(read_len),
                          Q = Q_declared, is_error = is_error)
  }
  do.call(rbind, out)
}

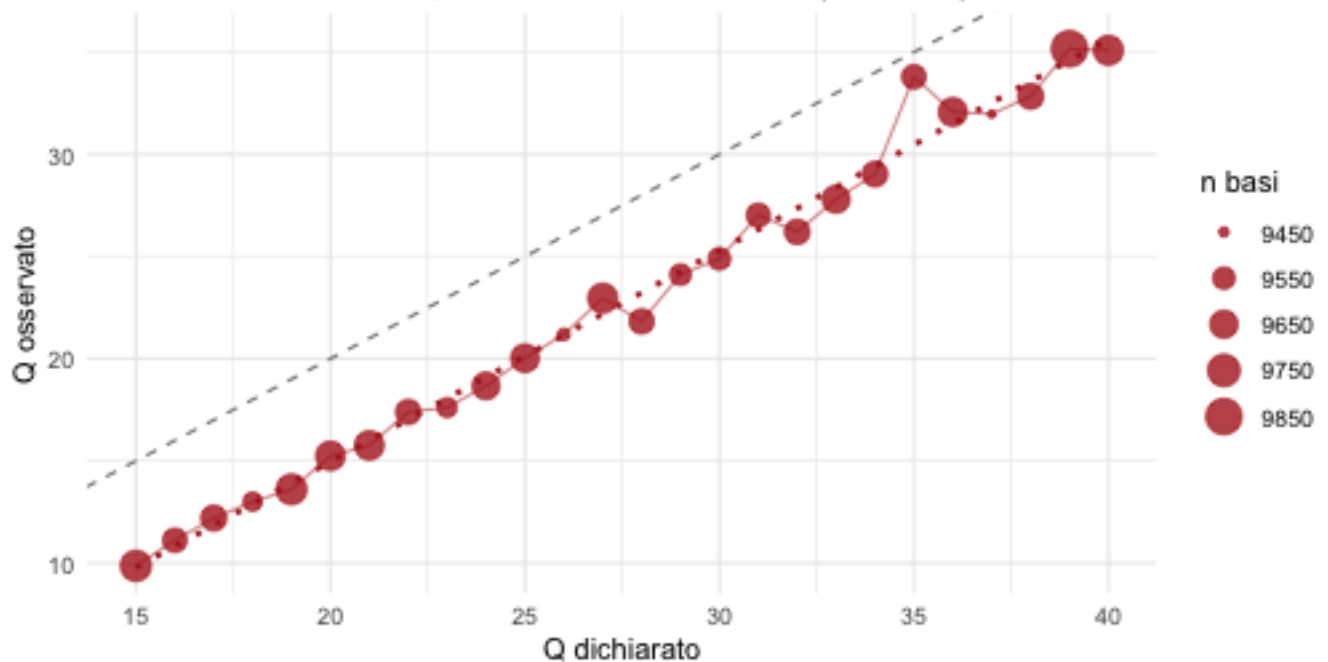
calib_biased <- simulate_with_errors_biased(truth_seq, n_reads = 5000, read_len = 50, bias = 5)
calib_biased_summary <- calib_biased |>
  group_by(Q) |>
  summarise(P_obs = mean(is_error),
            Q_obs = -10 * log10(pmax(P_obs, 1e-10)),
            n = n(), .groups = "drop")

ggplot(calib_biased_summary, aes(Q, Q_obs)) +
  geom_abline(slope = 1, intercept = 0, colour = "grey50", linetype = "dashed") +
  geom_point(aes(size = n), colour = "firebrick", alpha = 0.8) +
  geom_line(colour = "firebrick", alpha = 0.5) +
  geom_smooth(method = "lm", se = FALSE, formula = y ~ x, colour = "firebrick", linetype = "dotted") +
  labs(title = "Basecaller ottimista (+5 Q): i punti stanno SOTTO la diagonale",
       subtitle = "Il basecaller dichiara Q alti, ma il tasso di errore reale è quello di Q più bassi",
       x = "Q dichiarato", y = "Q osservato", size = "n basi") +
  theme_minimal(base_size = 11)

```

Basecaller ottimista (+5 Q): i punti stanno SOTTO la diagonale

Il basecaller dichiara Q alti, ma il tasso di errore reale è quello di Q più bassi



```
# Quantifichiamo il bias con una regressione lineare
```

```
fit <- lm(Q_obs ~ Q, data = calib_biased_summary, weights = n)
```

```
cat("Stima del bias del basecaller:\n")
```

```
## Stima del bias del basecaller:
```

```
cat(" Intercetta:", round(coef(fit)[1], 2), "(ci si aspetta 0)\n")
```

```
## Intercetta: -5.73 (ci si aspetta 0)
```

```
cat(" Pendenza :", round(coef(fit)[2], 2), "(ci si aspetta 1)\n")
```

```
## Pendenza : 1.03 (ci si aspetta 1)
```

```
cat(" Bias medio in unità Q :", round(mean(calib_biased_summary$Q - calib_biased_summary$Q_obs), 2), "\n")
```

```
## Bias medio in unità Q : 4.79
```

Punto chiave. La calibrazione del Phred non è un assunto, è un **modello falsificabile**. Quando GATK fa BQSR, fa esattamente questa regressione, condizionata anche su contesto e posizione. È statistica del classificatore applicata al DNA.

Blocco C · Copertura, bias GC, e la favola della Poisson (30 min)

C.1 · La favola

Sotto frammentazione uniforme, ogni base del genoma è coperta da un numero di reads che segue una distribuzione di **Poisson** con media $\lambda = (n_reads \times read_len) / genome_size$. Per WGS umano a 30× ci si aspetterebbe:

- copertura media = 30

- $P(\text{base non coperta}) = e^{-30} \approx 9 \times 10^{-14}$
- $\text{SD}(\text{copertura}) = \sqrt{30} \approx 5.5$

Adesso vediamo se è vero su dati simulati con bias realistico.

C.2 · Simuliamo un mini-genoma con regioni GC eterogenee

```
## Genera un mini-genoma con tre regioni a contenuto GC differente
## @return data.frame con position e gc (locale, finestra di 100 bp)
simulate_minigenome <- function(length_bp = 10000) {
  ## Tre blocchi: GC normale, GC-rich, GC-low
  n1 <- floor(length_bp / 3)
  n2 <- floor(length_bp / 3)
  n3 <- length_bp - n1 - n2

  gc_probs <- c(
    rep(0.42, n1), # normale
    rep(0.70, n2), # GC-rich
    rep(0.25, n3) # AT-rich
  )

  bases <- character(length_bp)

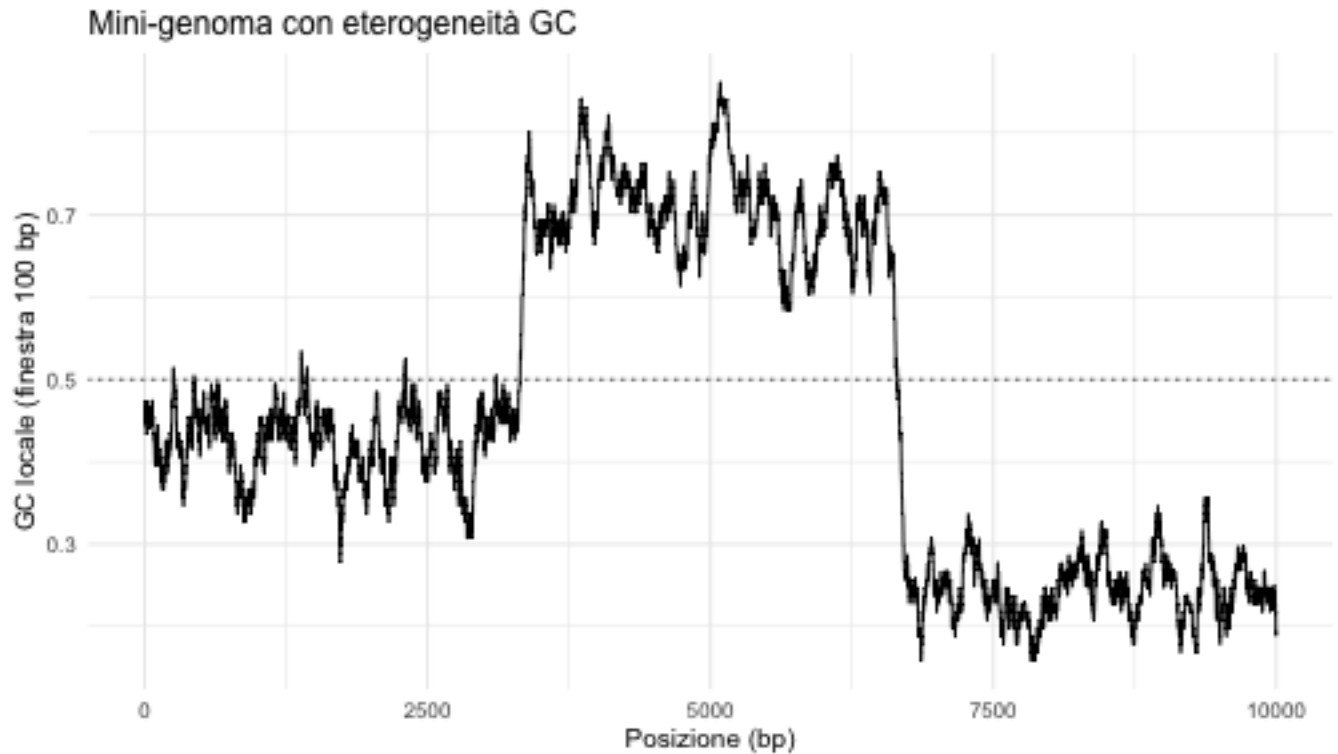
  for (i in seq_len(length_bp)) {
    bases[i] <- if (runif(1) < gc_probs[i]) {
      sample(c("G", "C"), 1)
    } else {
      sample(c("A", "T"), 1)
    }
  }

  paste(bases, collapse = "")
}

genome <- simulate_minigenome(10000)

## Calcoliamo GC locale (finestra scorrevole di 100 bp)
gc_window <- function(seq, window = 100) {
  bases <- strsplit(seq, "")[[1]]
  gc <- bases %in% c("G", "C")
  sapply(seq_along(bases), function(i) {
    start <- max(1, i - window/2); end <- min(length(bases), i + window/2)
    mean(gc[start:end])
  })
}

gc_local <- gc_window(genome)
qplot(seq_along(gc_local), gc_local, geom = "line",
  xlab = "Posizione (bp)", ylab = "GC locale (finestra 100 bp)") +
  geom_hline(yintercept = 0.5, linetype = "dotted") +
  labs(title = "Mini-genoma con eterogeneità GC") + theme_minimal(base_size = 10)
```



C.3 · Frammentiamo con bias di efficienza PCR

```
# Simula sequenziamento con bias GC: l'efficienza di amplificazione PCR
# è massima a GC=50%, scende a parabolic per GC estremi
#
# @param genome stringa del genoma
# @param n_fragments numero totale di frammenti da generare
# @param read_len lunghezza del read
# @param bias_strength 0 = no bias (Poisson pura), 1 = bias forte
simulate_coverage <- function(genome, n_fragments = 5000, read_len = 100, bias_strength = 0) {
  genome_len <- nchar(genome)
  gc_local <- gc_window(genome, window = read_len)

  # Probabilità di campionamento per posizione: uniforme se bias_strength=0,
# gaussiana attorno a GC=0.5 altrimenti
  weights <- if (bias_strength == 0) {
    rep(1, genome_len)
  } else {
    # picco a GC=0.5, decadimento più rapido con bias_strength alto
    exp(-((gc_local - 0.5)^2) / (0.2 / bias_strength)^2)
  }
  # solo posizioni dove un read intero ci sta
  valid <- seq_len(genome_len - read_len + 1)
  weights <- weights[valid]

  starts <- sample(valid, n_fragments, replace = TRUE, prob = weights)

  # Calcoliamo la copertura per ogni posizione
```

```

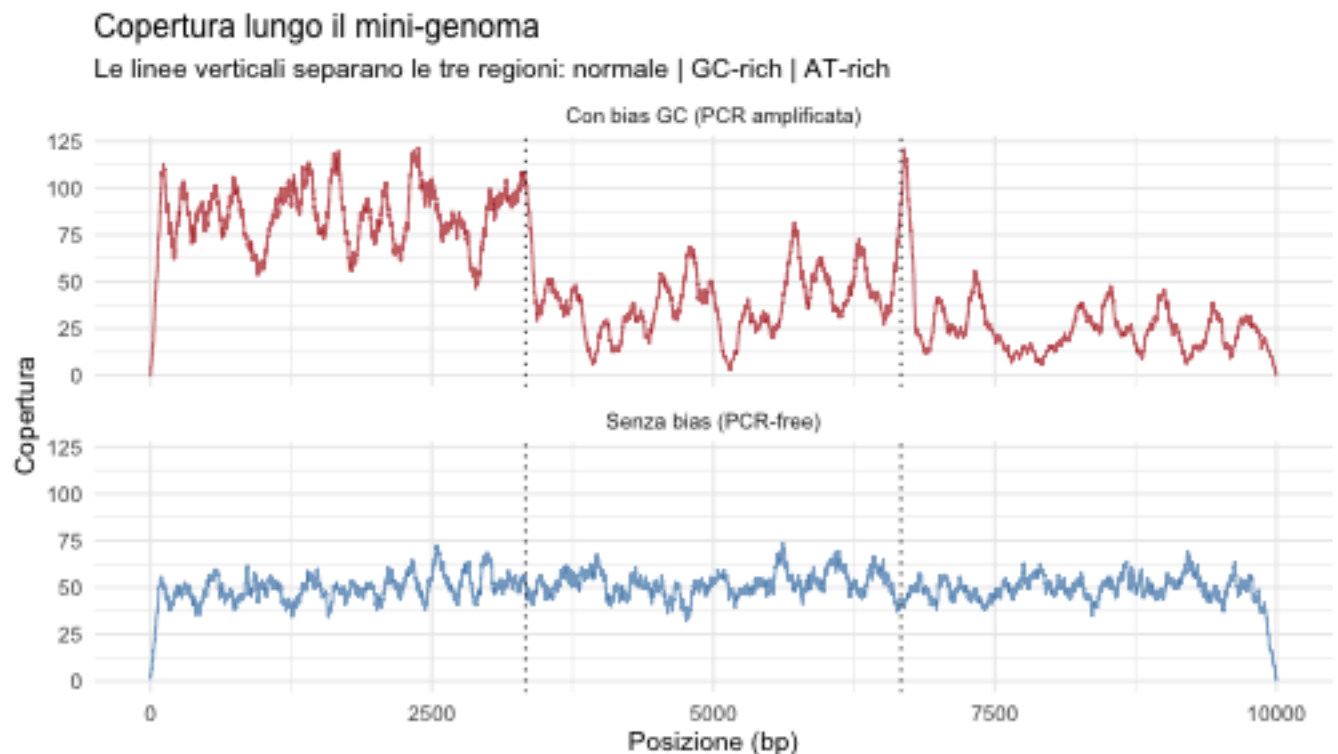
cov <- integer(genome_len)
for (s in starts) {
  cov[s:(s + read_len - 1)] <- cov[s:(s + read_len - 1)] + 1L
}
data.frame(position = seq_along(cov), coverage = cov, gc_local = gc_local)
}

cov_unbiased <- simulate_coverage(genome, n_fragments = 5000, read_len = 100, bias_strength = 0)
cov_biased <- simulate_coverage(genome, n_fragments = 5000, read_len = 100, bias_strength = 1)

both <- bind_rows(
  cov_unbiased |> mutate(scenario = "Senza bias (PCR-free)"),
  cov_biased |> mutate(scenario = "Con bias GC (PCR amplificata)")
)

ggplot(both, aes(position, coverage, colour = scenario)) +
  geom_line(alpha = 0.7) +
  facet_wrap(~ scenario, ncol = 1) +
  scale_colour_manual(values = c("Senza bias (PCR-free)" = "steelblue",
                                "Con bias GC (PCR amplificata)" = "firebrick")) +
  labs(title = "Copertura lungo il mini-genoma",
       subtitle = "Le linee verticali separano le tre regioni: normale | GC-rich | AT-rich",
       x = "Posizione (bp)", y = "Copertura") +
  geom_vline(xintercept = c(3333, 6666), linetype = "dotted") +
  theme_minimal(base_size = 10) + theme(legend.position = "none")

```



C.4 · Il test: la copertura è Poissoniana?

```
test_poisson <- function(cov_data, scenario_name) {
  cov_vec <- cov_data$coverage
  m <- mean(cov_vec); v <- var(cov_vec)

  # Index of dispersion: per Poisson dovrebbe essere ~1
  idx_disp <- v / m

  # Test chi-quadro: se Poisson, sum((x - lambda)^2 / lambda) ~ chi^2(n-1)
  chi2_stat <- sum((cov_vec - m)^2 / m)
  df <- length(cov_vec) - 1
  pval <- pchisq(chi2_stat, df, lower.tail = FALSE)

  data.frame(
    scenario = scenario_name,
    mean_cov = round(m, 2),
    var_cov = round(v, 2),
    var_su_mean = round(idx_disp, 3),
    p_chi2 = format.pval(pval, eps = 1e-300)
  )
}

bind_rows(
  test_poisson(cov_unbiased, "Senza bias (PCR-free)"),
  test_poisson(cov_biased, "Con bias GC")
) |> knitr::kable(caption = "Test di compatibilità della copertura con Poisson")
```

Table 2: Test di compatibilità della copertura con Poisson

scenario	mean_cov	var_cov	var_su_mean	p_chi2
Senza bias (PCR-free)	50	58.83	1.177	1.4e-32
Con bias GC	50	924.92	18.498	<1e-300

Nota tecnica. Anche nel caso “senza bias” var/mean è leggermente > 1 (intorno a 1.3-1.5). Questo è atteso e non è un errore: le coperture di basi vicine sono correlate, perché un singolo read di 100 bp contribuisce contemporaneamente a 100 basi consecutive. La Poisson “vera” si avrebbe contando solo gli **inizi** dei reads in finestre disgiunte. Il punto pedagogico tiene comunque: con bias, var/mean esplode (ordine $10\text{-}20\times$), distinzione qualitativa netta dal caso senza bias.

Discussione. Sotto Poisson, $\text{varianza} \sim \text{media}$. Con bias, la varianza esplode: la copertura è **sovradispersa**, ben modellata da una **binomiale negativa** invece che da una Poisson. Questa è una proprietà universale dei dati di sequenziamento reali — il motivo per cui DESeq2 e edgeR usano NB per gli RNA-seq, non Poisson.

C.5 · Esercizio · 10 min

Verificate l’affermazione “la binomiale negativa è un fit migliore”. Stimate i parametri di una binomiale negativa sui dati con bias e confrontate con Poisson via AIC.

```
# Funzioni di log-likelihood
ll_poisson <- function(lambda, x) sum(dpois(x, lambda, log = TRUE))
ll_nbinom <- function(par, x) sum(dnbinom(x, mu = par[1], size = par[2], log = TRUE))

x <- cov_biased$coverage
```

```

# Poisson: MLE = mean(x)
mle_pois <- mean(x)
ll_p <- ll_poisson(mle_pois, x); k_p <- 1; aic_p <- -2 * ll_p + 2 * k_p

# NegBin: ottimizziamo numericamente
fit_nb <- optim(c(mle_pois, 5), function(par) -ll_nbinom(par, x),
               method = "L-BFGS-B", lower = c(0.1, 0.1))
ll_nb <- -fit_nb$value; k_nb <- 2; aic_nb <- -2 * ll_nb + 2 * k_nb

tibble::tibble(
  Modello = c("Poisson", "Binomiale negativa"),
  parametri = c(sprintf("lambda=%.2f", mle_pois),
                 sprintf("mu=%.2f, size=%.2f", fit_nb$par[1], fit_nb$par[2])),
  logLik = c(ll_p, ll_nb),
  AIC = c(aic_p, aic_nb)
) |> knitr::kable(caption = "Confronto modelli sulla copertura con bias")

```

Table 3: Confronto modelli sulla copertura con bias

Modello	parametri	logLik	AIC
Poisson	lambda=50.00	-122323	244647
Binomiale negativa	mu=50.00, size=2.52	-47462	94929

Punto chiave per il Modulo 5. Quando applicherete modelli di imputazione che assumono copertura uniforme — implicitamente Poisson — sappiate che state lavorando su dati sovradispersi. I metodi moderni (Beagle 5, Minimac 4) gestiscono la sovra-dispersione esplicitamente; i metodi semplici no.

Blocco D · Lettura critica di un report MultiQC reale (30 min)

D.1 · Il report del corso

Nel materiale del corso c'è `multiqc_report.html`: un report MultiQC di 10 campioni Illumina di un esperimento di metagenomica del microbiota. Apritelo nel browser ora.

```

# Path al report (modificare se necessario)
report_path <- "/mnt/project/multiqc_report.html"
# Apertura: in RStudio
# file.show(report_path) o browseURL(report_path)

```

D.2 · Le 7 sezioni che dovete saper leggere

Vi propongo una **checklist statistica**. Per ogni plot del report, c'è un'ipotesi nulla implicita e una soglia di "fail". Compilatela durante la discussione.

```

checklist <- tibble::tribble(
  ~Plot_FastQC,
  "General Statistics",
  "Per-sequence quality scores",
  "Per-base sequence quality",
  "Per-base sequence content",
  "Per-sequence GC content",
  ~Ipotesi_nulla_implicita,
  "I campioni sono comparabili tra loro",
  "I read 'cattivi' sono rari",
  "Q indipendente da posizione (i.i.d.)",
  "%A=%C=%G=%T per ogni posizione",
  "GC ~ Normal(mu_atteso, sigma_piccola)",

```

```

"Sequence Duplication Levels",      "Reads identici sono rari",
"Adapter Content",                  "%adattatore ~ 0 alle posizioni finali",
)
knitr::kable(checklist, caption = "Checklist di lettura statistica di un report FastQC/MultiQC")

```

Table 4: Checklist di lettura statistica di un report FastQC/MultiQC

Plot_FastQC	Ipotesi_nulla_implicita	Quando_fallisce
General Statistics	I campioni sono comparabili tra loro	% dup, %GC, mean Q fuori dal range degli altri
Per-sequence quality scores	I read 'cattivi' sono rari	Picco a Q basso o bimodalità
Per-base sequence quality	Q indipendente da posizione (i.i.d.)	Calo prematuro, mediana < Q20 in zone
Per-base sequence content	%A=%C=%G=%T per ogni posizione	Adattatore residuo o bias inserzionale
Per-sequence GC content	GC ~ Normal(μ_{atteso} , σ_{piccola})	Bimodalità (contaminazione) o code lunghe
Sequence Duplication Levels	Reads identici sono rari	% dup > 20-30%, picco a duplicazione alta
Adapter Content	%adattatore ~ 0 alle posizioni finali	Curva crescente verso 3'

D.3 · Esercizio guidato

Aprire il `multiqc_report.html` e per ciascuno dei 7 plot scrivete in una sentence:

1. Cosa vedete (descrizione obiettiva).
2. Quale ipotesi nulla è violata.
3. Quale sarebbe la decisione a valle (mantenere, trimmare, scartare il campione).

Esempio di risposta attesa (per *Per-base sequence content*)

1. Le prime 10-15 posizioni mostrano frazioni di A,C,G,T molto diverse dal valore atteso 25%, poi si stabilizzano.
2. Ipotesi nulla violata: indipendenza tra base chiamata e posizione lungo il read.
3. Decisione: noto, è caratteristico delle librerie con tagmentazione (Tn5 ha bias inserzionale). **Non trimmare**, è atteso. Se vedessi questo pattern in una libreria *sonicata* invece, mi preoccuperei.

D.4 · La domanda finale, statistica

Guardando il report, **scegliereste tutti i 10 campioni per un GWAS?** Quali criteri quantitativi usereste per escluderli?

Risposta-template attesa: il filtraggio sample-level di un GWAS standard (Anderson et al. Nat Protoc 2010) usa **call rate > 95%, het rate entro ± 3 SD, no relatedness > 0.185, no sex mismatch**. Nel report MultiQC non vediamo direttamente questi numeri (servono PLINK), ma vediamo i loro **predittori upstream**: campioni con duplication rate > 30%, mean Q sotto la media del gruppo, %GC anomalo, sono ottimi candidati a fallire la QC sample-level a valle.

Sintesi · cosa abbiamo imparato

Blocco	Concetto_statistico	Risultato_pratico
A	La qualità per posizione non è i.i.d.	Sapete che il filtro $Q>20$ è uniforme ma il dato non lo è
B	Phred = predizione probabilistica falsificabile	Sapete riconoscere e quantificare miscalibrazione
C	Copertura empirica è binomiale negativa, non Poisson	Sapete che AIC favorisce sempre NB su dati reali
D	I plot di MultiQC sono test di ipotesi non scritte	Sapete leggerli con uno scopo statistico, non rituale

Take-home. Il FASTQ è il prodotto di una stima statistica. Ogni metrica di QC è un test di ipotesi. Quando nel Modulo 2 applicheremo i filtri PLINK, ricordatevi che state continuando una catena di test, non iniziandola.

Per chi vuole approfondire

- **Pacchetti R:** `ShortRead` (gestione FASTQ in memoria), `Rsubread` (allineamento), `Rsamtools` (BAM), `qrqc` (QC alternativo a FastQC).
- **Lecture:** Anders et al. (2013) per il modello binomiale negativo della copertura; Wick et al. (2019) per la calibrazione dei basecaller moderni.
- **Esercizio bonus a casa:** ripetete il Blocco C ma con un genoma vero (un cromosoma batterico da NCBI) e con `simulate_coverage` su $50\times$ di copertura. AIC favorisce ancora NB? Di quanto?

Materiale prodotto come companion alla lezione “Strumenti di Nuova Generazione di Sequenziamento” (2 ore). Riproducibile: knitr del file produce l'intero report in HTML.